

# ESPECIFICAÇÃO ARQUITETURAL – APLICAÇÃO WEB MVC

**Objetivo:** Mapear a arquitetura de uma aplicação web no lado do servidor (Back-end), utilizando o padrão MVC e os mecanismos do protocolo HTTP.

**Cenário:** A aplicação legada será reorganizada para separar responsabilidades em Model, View e Controller, facilitando manutenção, segurança, testes e evolução do sistema.

## 1. ARQUITETURA MVC E FLUXO DE AUTENTICAÇÃO

O padrão MVC divide a aplicação em três responsabilidades principais. A **View** apresenta a interface HTML e coleta as credenciais. O **Controller** recebe a requisição HTTP, interpreta os parâmetros e coordena o processo. O **Model** representa os dados e regras de negócio, incluindo a consulta e validação do usuário.

| Navegador<br>HTML / formulário           | Controller<br>Servlet / LoginController | Model<br>User / AuthService / DAO           |
|------------------------------------------|-----------------------------------------|---------------------------------------------|
| 1. POST /login<br>usuário + senha        | 2. recebe Request<br>valida entrada     | 3. consulta usuário<br>e valida credenciais |
| 6. recebe Response<br>e cookie de sessão | 5. cria sessão<br>e redireciona         | 4. retorna resultado<br>válido/inválido     |

**Fluxo:** o cliente envia uma requisição HTTP POST para o endpoint de autenticação. O Servlet/Controller valida os dados, aciona o Model para localizar o usuário e verificar a senha. Em caso de sucesso, o servidor cria uma sessão, associa o usuário autenticado a ela e devolve uma resposta HTTP com um cookie de identificação de sessão. Nas requisições seguintes, o navegador envia esse cookie e o servidor recupera a sessão correspondente.

## 2. MAPEAMENTO DOS COMPONENTES

**View (JSP/HTML):** formulário de login, mensagens de erro e páginas protegidas. A View não deve acessar diretamente o banco de dados nem conter regras de autenticação.

**Controller (Servlets):** recebe GET/POST, valida parâmetros, chama os serviços do Model, controla redirecionamentos e decide qual View será apresentada. Exemplos: *LoginServlet*, *LogoutServlet* e *DashboardServlet*.

**Model:** concentra entidades e regras de negócio. Pode conter *User*, *AuthService* e um DAO/repositório responsável pelo acesso ao banco. A senha deve ser armazenada de forma segura, utilizando hash apropriado, nunca em texto puro.

**HTTP:** o login utiliza normalmente POST porque envia credenciais no corpo da requisição. O servidor responde com códigos como 200, 302 ou 401, conforme o fluxo adotado. Páginas protegidas devem verificar a existência de uma sessão válida antes de liberar o conteúdo.

## 3. CONTROLE DE SESSÃO E COOKIES

HTTP é um protocolo essencialmente sem estado: cada Request/Response é independente. Para manter o usuário autenticado entre requisições, será utilizado o mecanismo de **sessão no servidor**. Após autenticação válida, o servidor cria uma sessão e gera um identificador único. Esse identificador é enviado ao cliente por meio de um cookie, normalmente associado ao mecanismo de sessão do framework/servidor.

Nas requisições posteriores, o navegador envia automaticamente o cookie. O servidor utiliza o identificador para localizar a sessão e recuperar atributos como o identificador do usuário, permissões e horário de expiração. Assim, a autenticação permanece válida durante o ciclo de navegação sem que a senha precise ser reenviada.

**Exemplo conceitual:** POST /login → autenticação → criação da sessão → Set-Cookie → GET /dashboard + Cookie → servidor recupera sessão → resposta autorizada.

## 4. PARECER TÉCNICO SOBRE A GESTÃO DE SESSÕES

A gestão de sessões no servidor é recomendada para esta aplicação porque permite separar as credenciais permanentes do estado temporário de autenticação. O navegador mantém apenas o identificador da sessão, enquanto os dados de autenticação permanecem sob controle do servidor. Essa abordagem reduz a exposição de informações sensíveis e facilita a invalidação da autenticação no logout.

O cookie de sessão deve ser configurado com **HttpOnly**, impedindo acesso ao cookie por JavaScript, e **Secure**, para que seja transmitido somente em HTTPS. Recomenda-se também **SameSite** apropriado para reduzir riscos de CSRF e um tempo de expiração compatível com a política de segurança da aplicação. Após o login, o identificador de sessão deve ser renovado para reduzir o risco de session fixation.

No logout, o Controller deve invalidar a sessão no servidor e orientar a remoção/expiração do cookie. Para áreas restritas, cada requisição deve passar por uma verificação de autenticação antes de acessar recursos protegidos. Caso a sessão seja inexistente ou expirada, o usuário deve ser redirecionado para a tela de login.

Essa estratégia está alinhada aos mecanismos de controle de estado normalmente apresentados em materiais de desenvolvimento web, como os capítulos sobre sessão e cookies de Basham ou Kurniawan: o cookie identifica o cliente, enquanto o estado da sessão é mantido e administrado pelo servidor.

## 5. FLUXO HTTP DETALHADO

| Etapa | Método / mecanismo | Comportamento                                           |
|-------|--------------------|---------------------------------------------------------|
| 1     | GET /login         | Servidor entrega a View do formulário.                  |
| 2     | POST /login        | Controller recebe usuário e senha.                      |
| 3     | Model              | AuthService/DAO consulta e valida as credenciais.       |
| 4     | Sessão             | Servidor cria a sessão e associa o usuário autenticado. |
| 5     | Set-Cookie         | Resposta envia o identificador da sessão ao navegador.  |
| 6     | GET /dashboard     | Navegador envia o cookie na requisição.                 |
| 7     | Validação          | Servidor recupera a sessão e verifica autorização.      |
| 8     | Logout             | Sessão é invalidada e o acesso protegido é encerrado.   |

## 6. CONCLUSÃO

A separação MVC proposta organiza a aplicação legada em camadas com responsabilidades claras: a View cuida da apresentação, o Controller coordena as requisições HTTP e o Model concentra regras e dados. O uso de sessão no servidor, identificada por cookie protegido, mantém o estado de autenticação entre requisições sem transferir credenciais continuamente. A arquitetura também favorece manutenção, testes, segurança e futura evolução da aplicação.

**Referências técnicas:** BASHAM, Bryan; SIERRA, Kathy; BATES, Bert. *Head First Servlets & JSP*. O'Reilly. KURNIAWAN, Budi. Materiais sobre desenvolvimento web Java, Servlets, sessões e cookies. Documentação técnica do protocolo HTTP e práticas de segurança web.